



Data Representation

- ① Decimal no. system (0-9) Base 10
- ② Binary no. system (0, 1) Base 2
- ③ Octal no. system (0-7) Base 8
- ④ Hexadecimal no. system (0-15) Base 16

1) Decimal to Binary
 $(97)_{10} \rightarrow (\quad)_2$

Remainder

2	97	1
2	48	0
2	24	0
2	12	0
2	6	0
2	3	1
	1	

$(1100001)_2$



How to check this is correct?

Binary to Decimal

$$1 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 +$$

$$0 \times 2^1 + 0 \times 2^0$$

$$\Rightarrow 64 + 32 + 0 + 0 + 0 + 0 + 1$$

$$= (97)_{10}$$

2) Decimal to Octal $(97)_{10} \rightarrow (\quad)_8$

@

8	97	1
8	12	4
	1	

$(141)_8$

3) Decimal to Hexadecimal
 $(97)_{10} \rightarrow (\quad)_{16}$



16	97	1
	6	

$(61)_{16}$

4) Octal to Decimal

$(141)_8 \rightarrow ()_{10}$

$$1 \times 8^2 + 4 \times 8^1 + 1 \times 8^0 \Rightarrow 64 + 32 + 1 = 97$$

5) Hexadecimal to Decimal

$(61)_{16} \rightarrow ()_{10}$

$$= 6 \times 16^1 + 1 \times 16^0$$

$$= 96 + 1$$

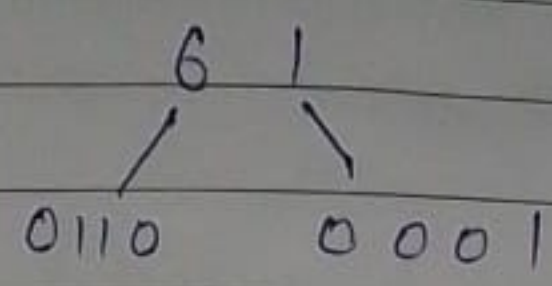
$$= 97$$

$(97)_{10}$

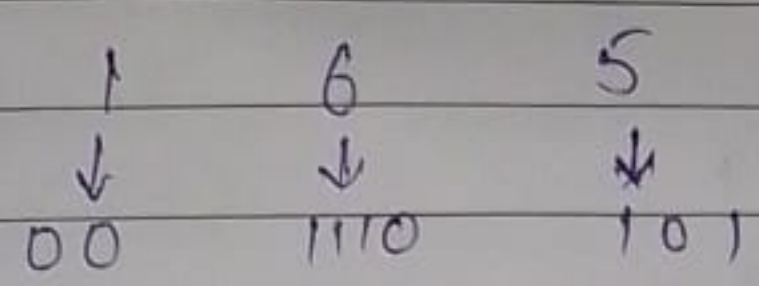
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

2

⑥ Hexadecimal to Binary
 $(61)_{16} \rightarrow (\quad)_2$



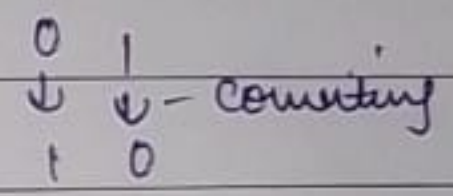
⑦ Octal to Binary
 $(165)_8 \rightarrow (\quad)_2$



Complement :- opposite

* 1's Complement

① $(1001)_2 \rightarrow (0110)_2$



② $(01001010)_2 \rightarrow (10110101)_2$

* 2's Complement

① $(101)_2$
 $\downarrow$
 $(010)_2$

② First convert ^{into} 1's complement then add 1.

```

  010
+   1
-----
 011  → (011)2
  
```

Fixed Point Representation - decimal part of a number



(3)

$$(10001)_2$$

 $\downarrow$

$$(01110)_2$$

$$01110$$

$$+ \quad \quad 1$$

$$\hline 01111$$

 $\rightarrow$

$$(01111)_2$$

* 9's complement

(1)

$$(873)$$

$$999$$

$$- \quad 873$$

$$\hline 126$$

(2)

$$(33815)$$

$$99999$$

$$- \quad 33815$$

$$\hline 66184$$

~~(3)~~

10's complement

(1)

$$34968$$

$$99999$$

$$- \quad 34968$$

$$\hline 65031$$

$$65031$$

$$+ \quad \quad 1$$

$$\hline 65032$$

• Fixed Point Representation $\rightarrow$ decimal point position fix



• Floating point Representation: - There is change in position of decimal point

* A real numbers in a computer can be represented in two ways: (given above)

1) Fixed Point Representation: - In this method, decimal point is placed in a fixed place, a number ending with decimal is called whole number and numbers starting with decimal point is called fractional number.

eg:- $45.00 \rightarrow$ whole number
 $\uparrow$ decimal extreme right

$00.45 \rightarrow$ fractional number
 $\downarrow$ decimal extreme left

2) Floating point Representation: - In this method, the position of decimal point is not specified by this a number can be expressed in many ways.

eg:- $4.35 \downarrow$

$$435 \times 10^0$$

$$\cancel{435 \times 10^0} \quad 43.5 \times 10^1$$

$$4.35 \times 10^2$$

$$.435 \times 10^3$$

Error detection and Error Correction

Sender

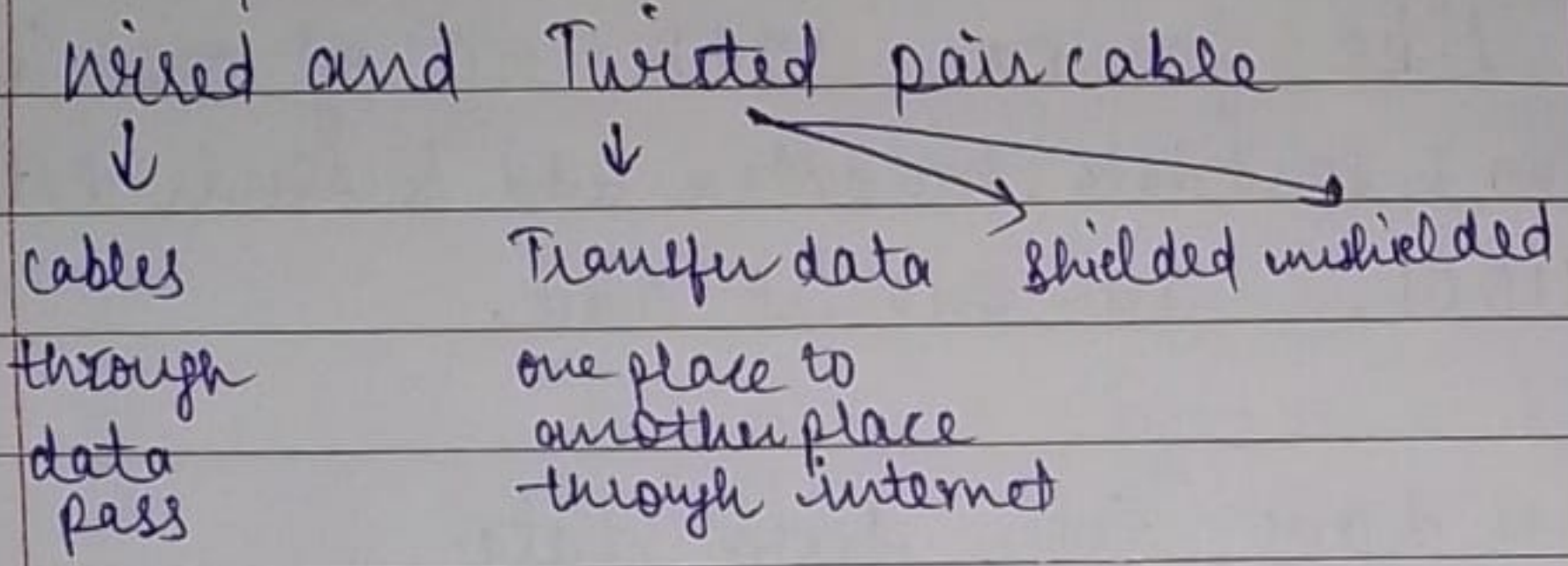
Receiver

1 0 1 0 1 0 1 0

1 0 1 0 1 1 1 0

- * Data Communication:- Sending or Receiving data, exchange data between two components.
- * Sender:- Sends the data
- * Receiver:- Receiving the data

Data paths:-

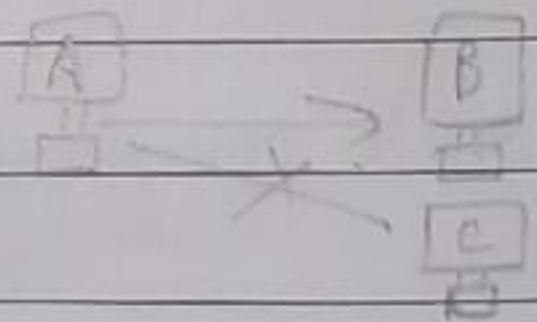


Wireless:- Radiowaves
Bluetooth
Microwaves
Infrared radi-

Fiber optics is fast data transfer.
(in the form of light)

* Protocol:- set of rules which help data communication.

- 1) What is communicated?
 - 2) How _____?
 - 3) When _____?
- Protocol defines these three way



Characteristics of data communication:-

- 1) Accuracy
- 2) Timeliness:- Data transfer at exact time which is given by user
- 3) jitter

Errors:- 1) One bit error (single bit error):- when in the data bit data is modified. (One bit changed)



2. 2-bit error :- When 2 bits are changed in data is called 2-bit error.

3. Burst error :- When data is changed in the sequence of more than 2 bits is called burst error.

Detection methods :-

1. Parity Check :- It is of two types

Even Parity

Odd Parity

1) Even Parity :- like for eg. 1011 no. of one is ~~even~~ there. there is odd no. & send data one extra add & send data.

2) Odd Parity :- Eg. 11001. no. one is odd.

2. Checksum :- we have given two data

Data word 1 $\rightarrow$ 1010 1001

Data word 2 $\rightarrow$ 0011 1001

First Add both data :- 1010 0010

One's complement :- 1110 0010

One's complement $\rightarrow$ 0001 1101 $\rightarrow$ It is
+ Checksum.

1111 1111

When comes one in data then there is no error.

3. Cycling Redundancy Check :- (CRC)

1010101010

(Data send kina ekda vich Redundant bit add krke)

$$x^4 + x^3 + 1$$

$$1x^4 + 1x^3 + 0x^2 + 0x^1 + 1x^0$$

$$(1 \quad 1 \quad 0 \quad 0 \quad 1)$$

Polynomial x - no. of bits
add 1

11001
← minus one no. of bits

X-OR
↓
Exclusive gate

$$\begin{array}{r}
 11001 \overline{) 10101010100000} \\
 \underline{11001} \\
 011000 \\
 \underline{11001} \\
 000011010 \\
 \underline{11001} \\
 000110000 \\
 \underline{11001} \\
 000010 \\
 \underline{11001} \\
 \text{XXXXXXXXXX}
 \end{array}$$

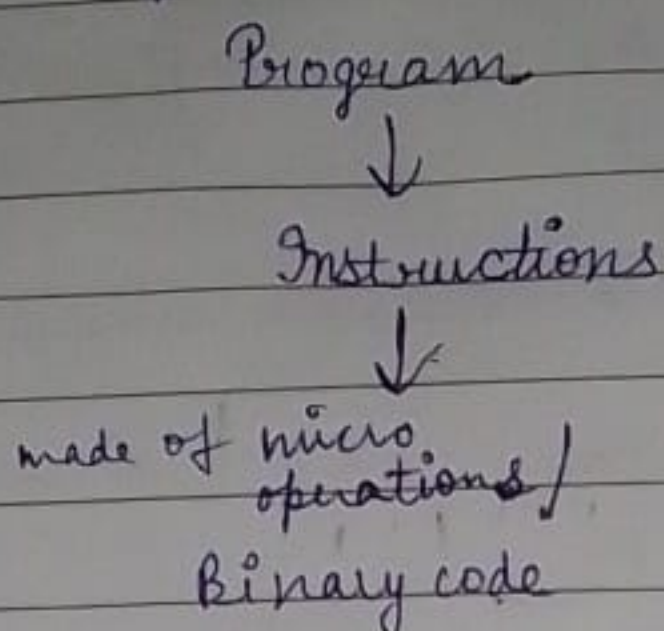
$$\begin{array}{r}
 11001 \overline{) 10101010100010} \\
 \underline{11001} \\
 011000 \\
 \underline{11001} \\
 000011010 \\
 \underline{11001} \\
 00011001 \\
 \underline{11001} \\
 000000
 \end{array}$$

$$m \rightarrow 10$$

$$r = 4$$

$$m + r = 10 + 4 = 14$$

Instruction Codes



Instruction set:- group of instructions

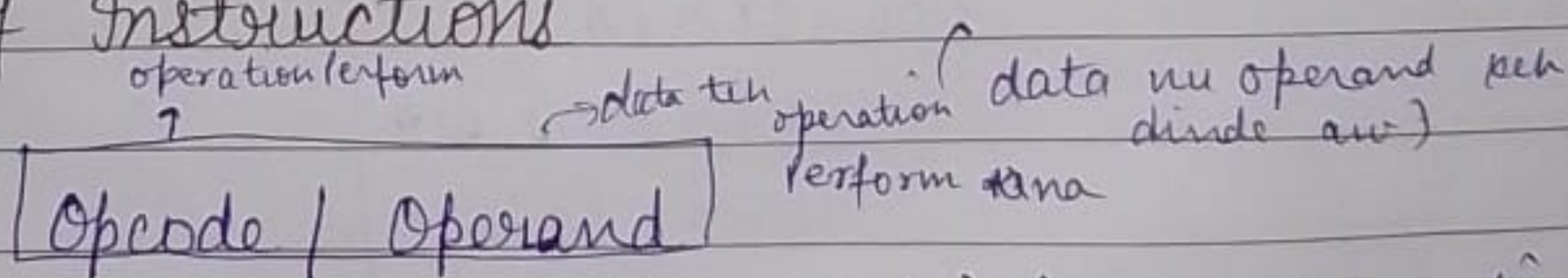
Data $\xrightarrow{\text{stores}}$ Memory
 Instruction $\xrightarrow{\text{stores}}$ Memory

ALU $\rightarrow$ Arithmetic logic operations
 actions $\rightarrow$ addition, subtraction, multiplication
 perform

AC $\rightarrow$ Accumulator $\rightarrow$ (Temporary storage)

Instruction code $\rightarrow$ set of bits, give instruction to computer to perform specific task.

Parts of Instructions

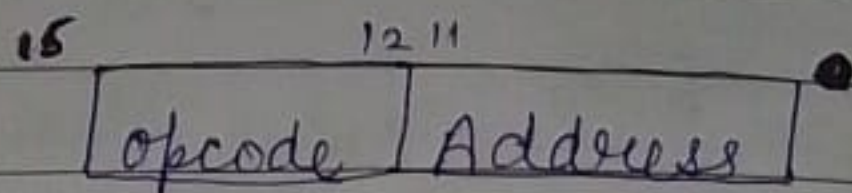


only operation performed like addition, subtraction, division etc is opcode.

operation performed on data i.e. data is called operand.



~~ADD~~ ADD 5
 ↓ ↓
 opcode operand



Instruction format

$2^{12} \rightarrow 4096$ memory

$2^4 \rightarrow 16$

$12-15 \rightarrow 2^4$ bits for opcode

16 bits memory for opcode

$0-10 \rightarrow 12$ bits used for address

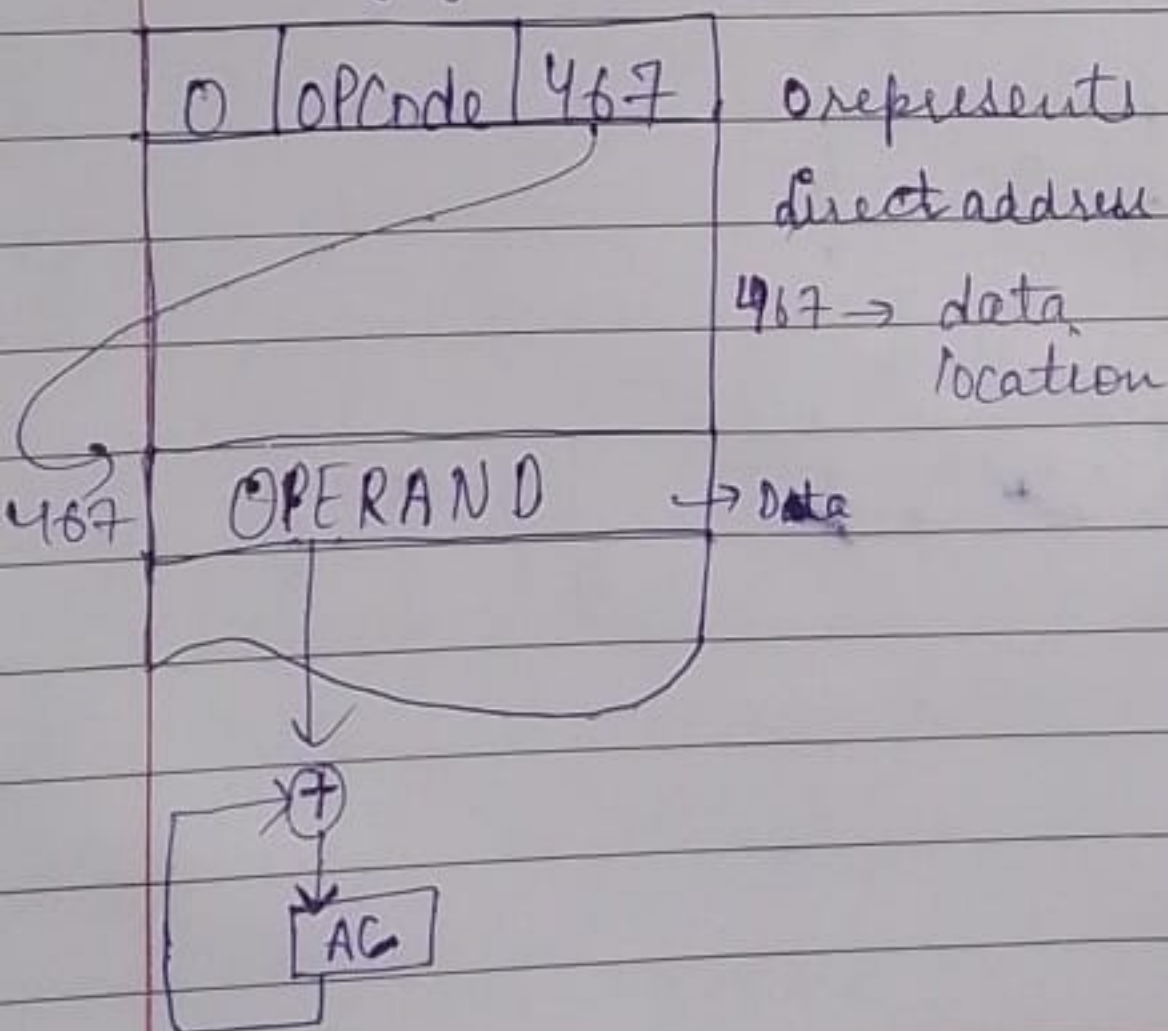
* Immediate Address

(Instruction deti naal data diinda)
 (data naal pyo hunda opcode de)

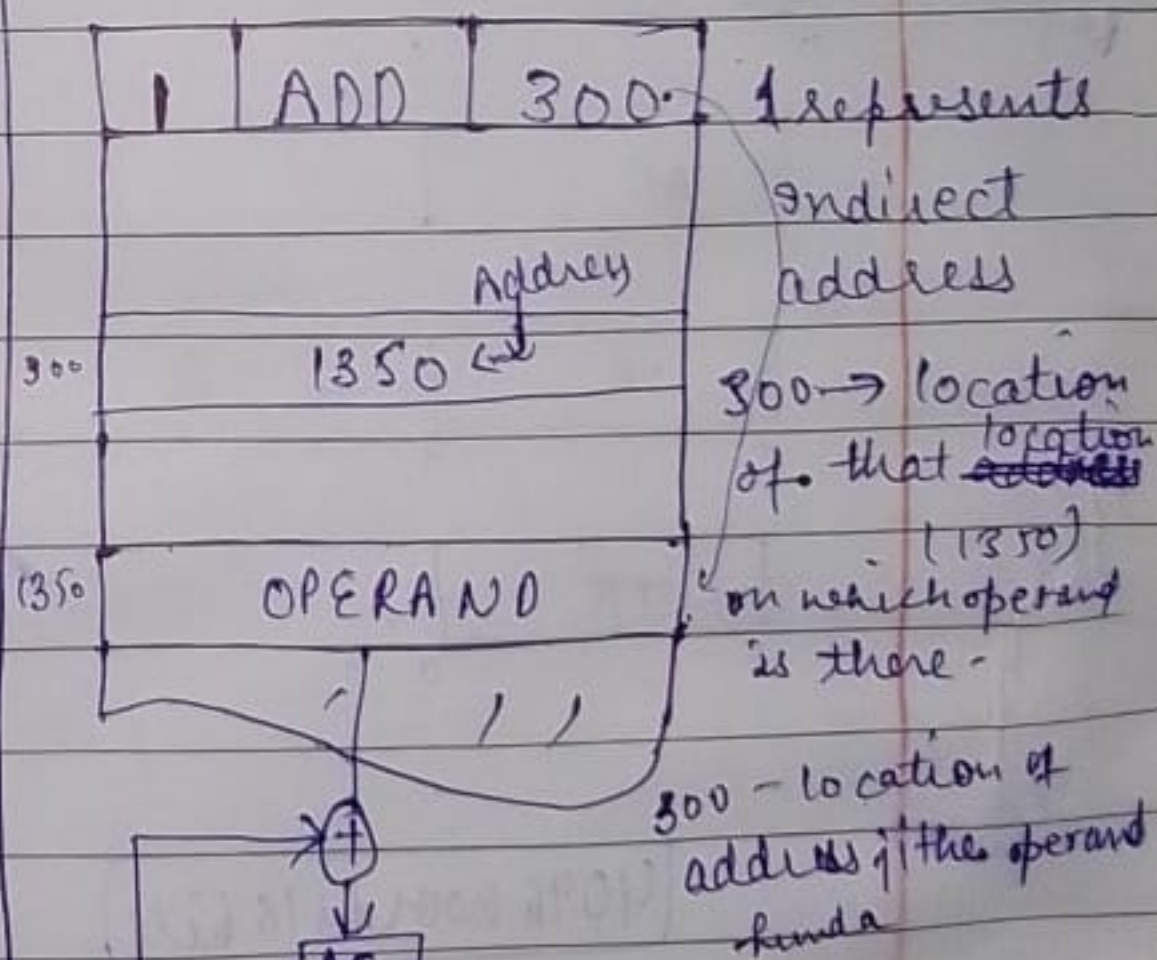
ADD 15

Operation per formed on data and their address is immediate there at that time.

Direct Address:- Data da address
 Pya hoaya jitha diinda data store aa.



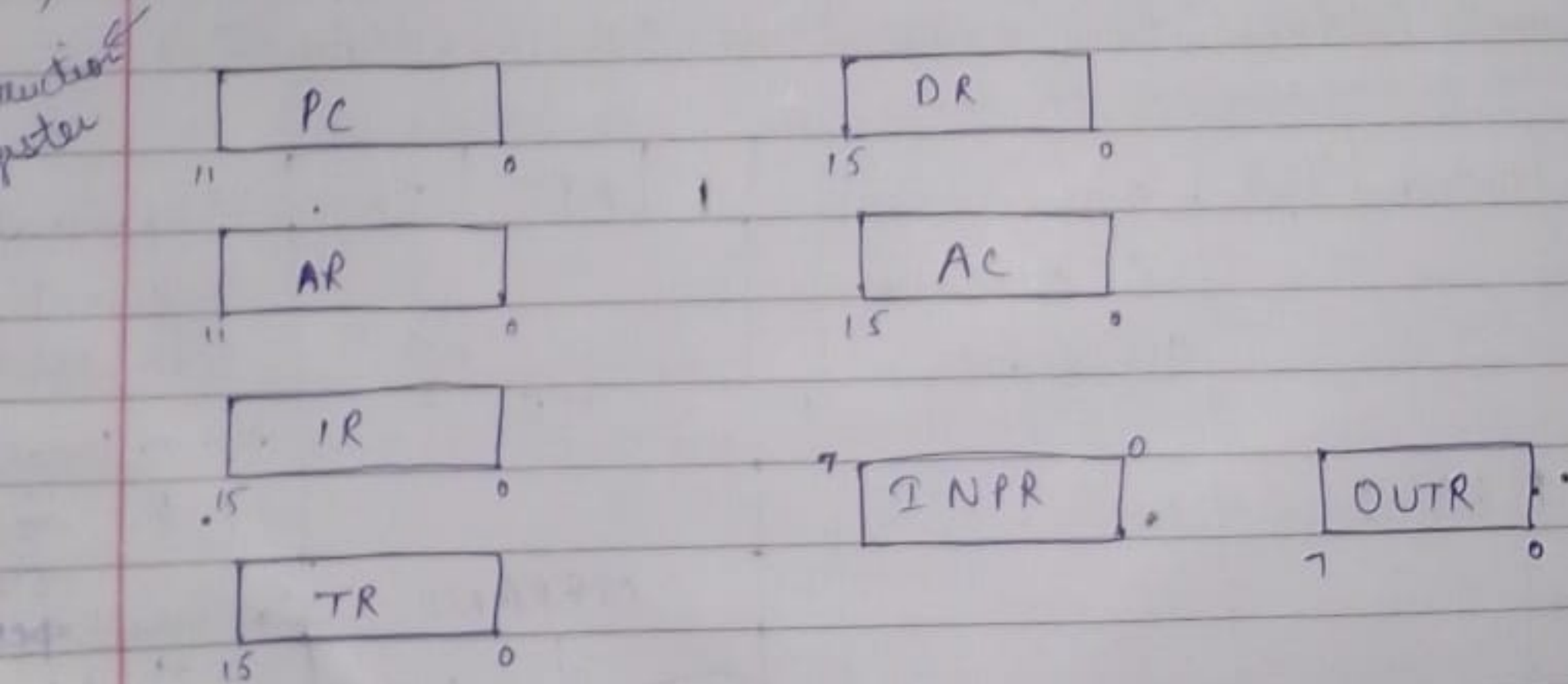
Indirect Address



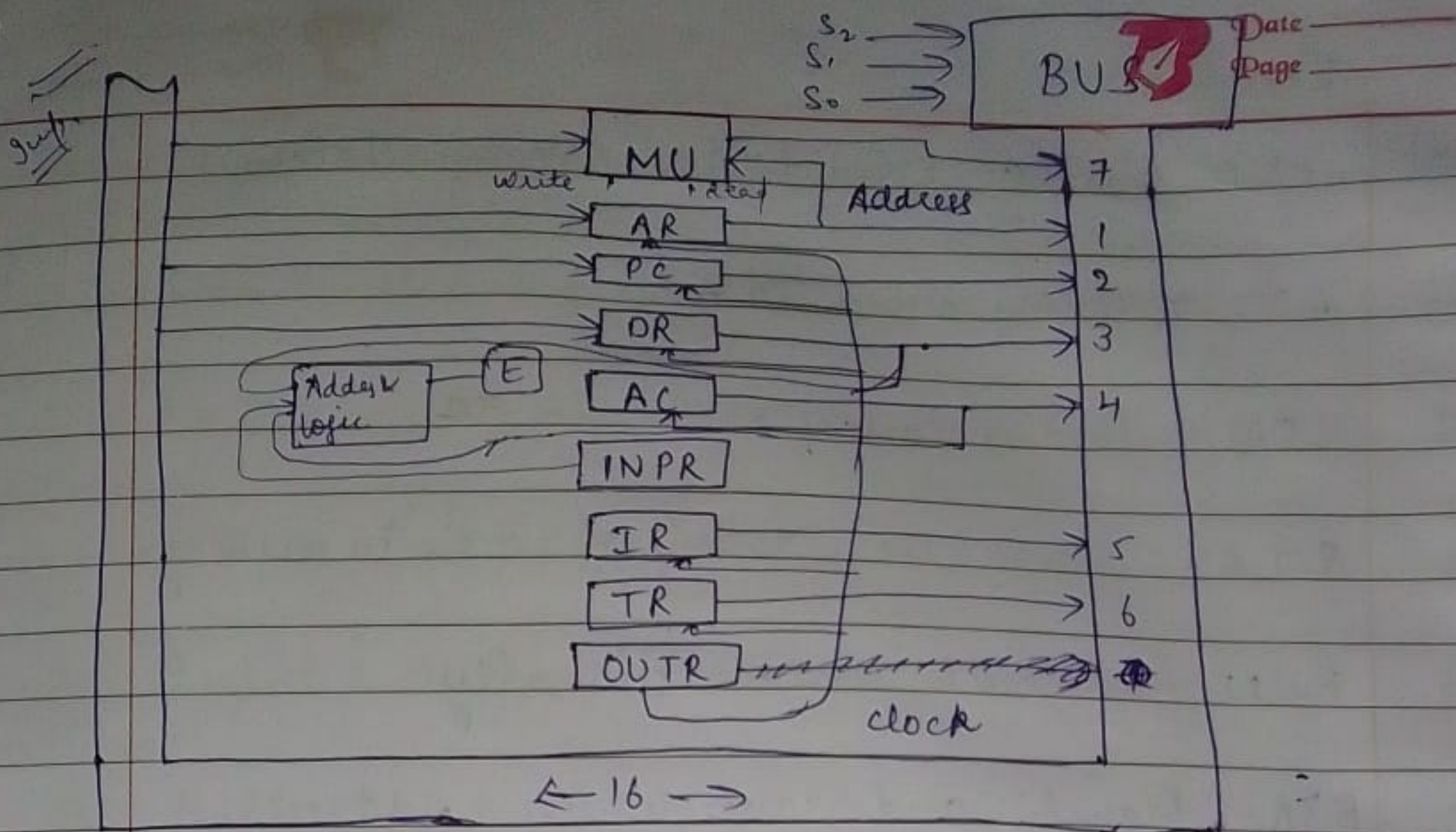


Computer Registers :-

- 1) DR $\rightarrow$ Data Register - 16 bit $\xrightarrow{\text{used to}}$ holds memory operand
- 2) AR $\rightarrow$ Address Register - 12 bit $\xrightarrow{\text{function}}$ holds address for memory
- 3) AC $\rightarrow$ Accumulator - 16 bit
(Processor Register)
- 4) PC $\rightarrow$ Program counter - 12 bits $\rightarrow$ holds the address of instruction
- 5) TR $\rightarrow$ Temporary Register - 16 bits $\rightarrow$ holds temporary data
- 6) INPR $\rightarrow$ Input Register - 8 bit $\rightarrow$ holds input character
- 7) OUPR $\rightarrow$ Output Register - 8 bits $\rightarrow$ holds output character
- 8) IR $\rightarrow$ 16 bits $\rightarrow$ holds the instruction code



4096 words 16 bits

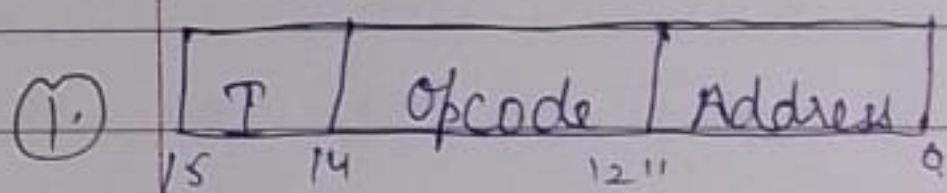


Computer Instructions

$I=0$ - Direct

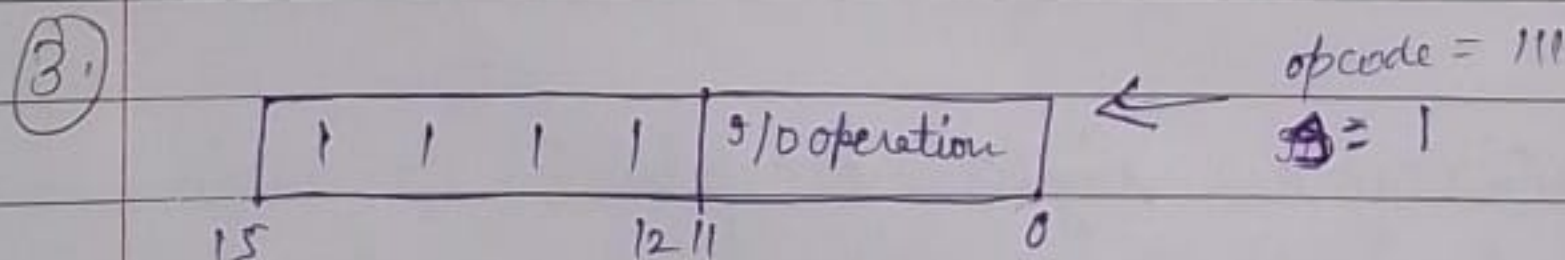
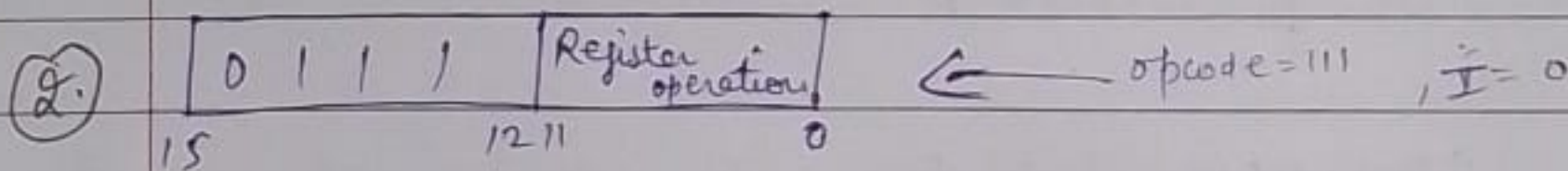
$I=1$ - Indirect

- 1.) Memory Reference - 16 bit
- 2.) Register Reference
- 3.) I/O



000
001
010
011
100
110

 Memory Reference Instructions



- 1.) Memory Reference

AND

$0 \times 0 = 0$
 $0 \times 1 = 0$
 $1 \times 0 = 0$
 $1 \times 1 = 1$

Examples :-



Date _____

Page _____

- # AND memory word to AC (accumulator)
- # ADD memory word to AC
- # LDA - Load memory word to AC
- # STA - Store the content of AC in memory
- # BUN - Branch unconditionally
- # BSA - Branch and save return address
- # ISZ - Increment and skip if zero.

2. Register Reference

- # CLA :- Clear AC.
- # CDE :- Clear E.
- # CMA :- Complement AC.
- # CME :- Complement E.
- # CIR :- Circular right AC & E.
- # CIL :- Circulate left AC & E.
- # INC :- Increment AC.
- # SPA :- Skip next instruction if AC is positive.



- # SNA:- Skip the next instruction if AC is negative.
- # SZA:- skip the next instruction if AC is zero.
- # SZE:- skip the next instruction if E is zero.
- # HLT:- ^(stop) Halt Computer.

3.) Input / Output:-

- # INP:- Input character to AC.
- # OUT:- Output character from AC.
- # SKI:- Skip ~~all~~ on input flag.
- # SKO:- Skip on output flag.
- # ION:- Interrupt on.
- # IOF:- Interrupt off.

Instruction Set completion:-

* A set of instructions are said to be complete if computer includes a sufficient number of instructions of following categories:-

- 1.) Arithmetic, logical and shift instructions.
- 2.) Instructions for moving information to and from memory and processor registers.



3.) Program control instruction together with instructions that check status conditions.

4.) Input and output instructions.

Timing & Control

master clock generator

1.) Hardwired control → fast operation

2.) Microprogrammend control

Q. ^{Ans.} What is difference b/w Hardwired & Microprogrammend?

Ans:-

1.) Implementation:-

Hardwired:- fixed set of logic gates and circuits

Microprogrammend:- Micro code stored in the memory

2.) Flexibility:-

Hardwired:- less flexible and difficult to modify.

Microprogrammend:- more flexible and easy to modify.

3.) Instruction Set:-

Hardwired:- Supports limited Instruction set

Microprogrammend:- Supports complex Instruction set

4. Complexity of Design:-

Hardwired:- Simple design, easy to implement.

Microprogrammend:- Complex design, difficult to implement.

5. Speed:-

Hardwired:- fast operation

Microprogrammend:- Slow operation, due to microcode decoding

6. Debugging and Testing:-

Hardwired:- Difficult to Debug and Test

Microprogrammend:- Easy to ^{Debug} ~~Debug~~ and Test

7. Size and Cost:-

Hardwired:- Smaller size, low cost.

Microprogrammend:- larger size, High cost.

8. Maintenance and upgradability

Hardwired:- Difficult to upgrade and maintain.

Microprogrammend:- Easy to upgrade and maintain.

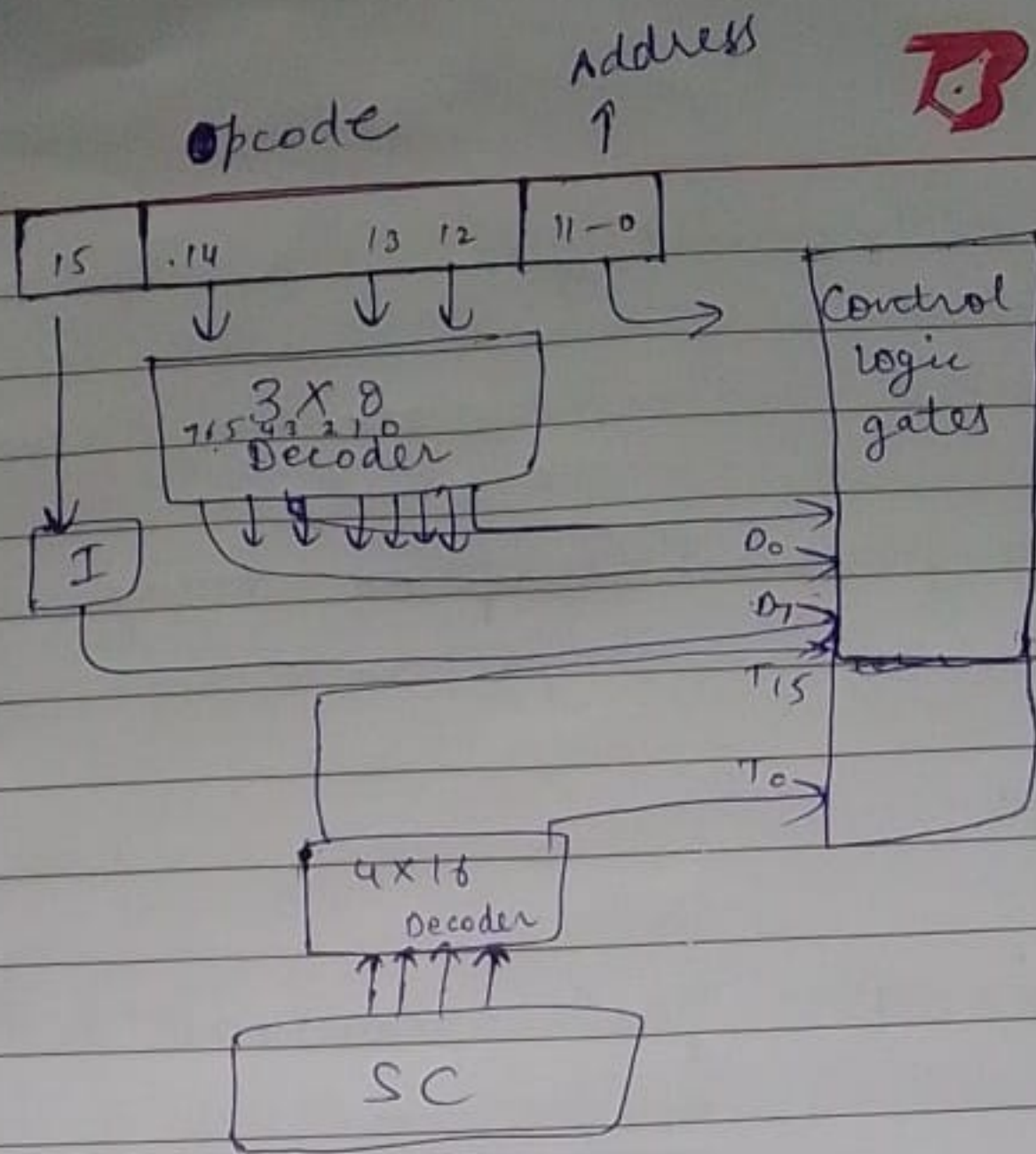


Diagram explanation

3 X 8 decoder $\xrightarrow{\text{means}}$ 3 Input, $2^3 \rightarrow 8$
 $\rightarrow$ 8 output produce $\xrightarrow{\text{hindiya}}$ control logic gates

4 X 16 decoder $\xrightarrow{\text{means}}$ 4 inputs kanda
16 output produce $\xrightarrow{\text{hindiya}}$ control logic gates

Instruction Cycle

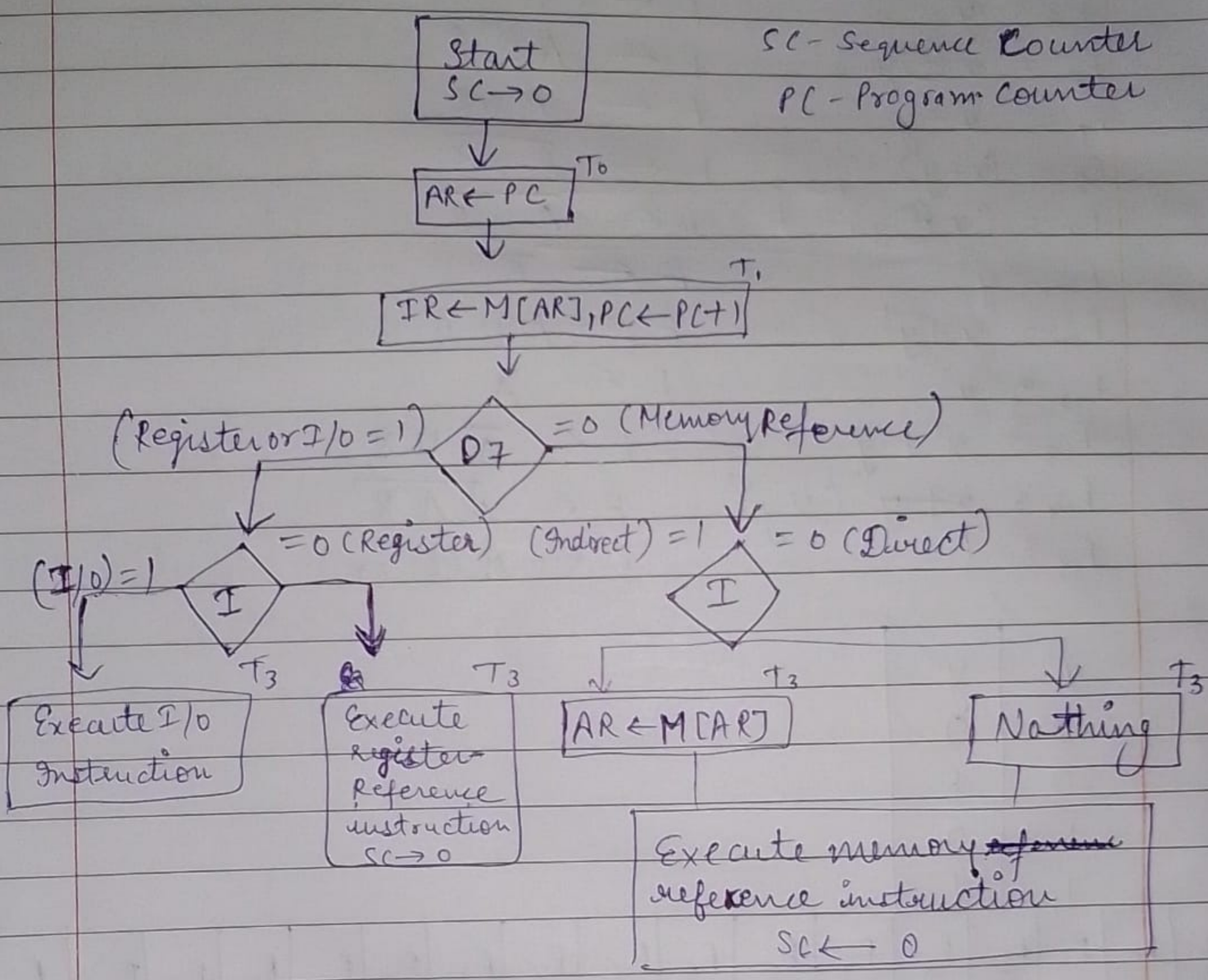
when instruction execute then which steps are taken

Instruction $\xrightarrow{\text{store}}$ memory

- 1.) Instruction $\rightarrow$ fetch ^(from) into memory
- 2.) $\rightarrow$ Decode
- 3.) $\rightarrow$ effective address
- 4.) $\rightarrow$ Execute

- 1.) Fetch the instruction from the memory.
- 2.) Decode the instruction
- 3.) Read the effective address from memory, If instruction has an indirect address
- 4.) Execute the instruction.

Flowchart



Microoperations :- four level operations made 16 logic operations

Λ - And ∨ - OR ⊕ - XOR	- → complement
---	-----------------------

No contents ↓

$$f \leftarrow 0$$

$$f \leftarrow A \wedge B$$

$$F \leftarrow A \wedge \overline{B}$$

$$f \hookrightarrow A$$

$$f \leftarrow \bar{A} \wedge B$$

$$f \leftarrow B$$

$$f \leftarrow A \oplus B$$

$$F \leftarrow A \vee B$$

$$f \leftarrow \overline{A \vee B}$$

$$f \leftarrow \overline{A \oplus B}$$

$$A \leftarrow \overline{B}$$

$$F \leftarrow A \vee \bar{B}$$

$$f \leftarrow \bar{A}$$

$$F \leftarrow \bar{A} \vee B$$

$$f \leftarrow \overline{A \wedge B}$$

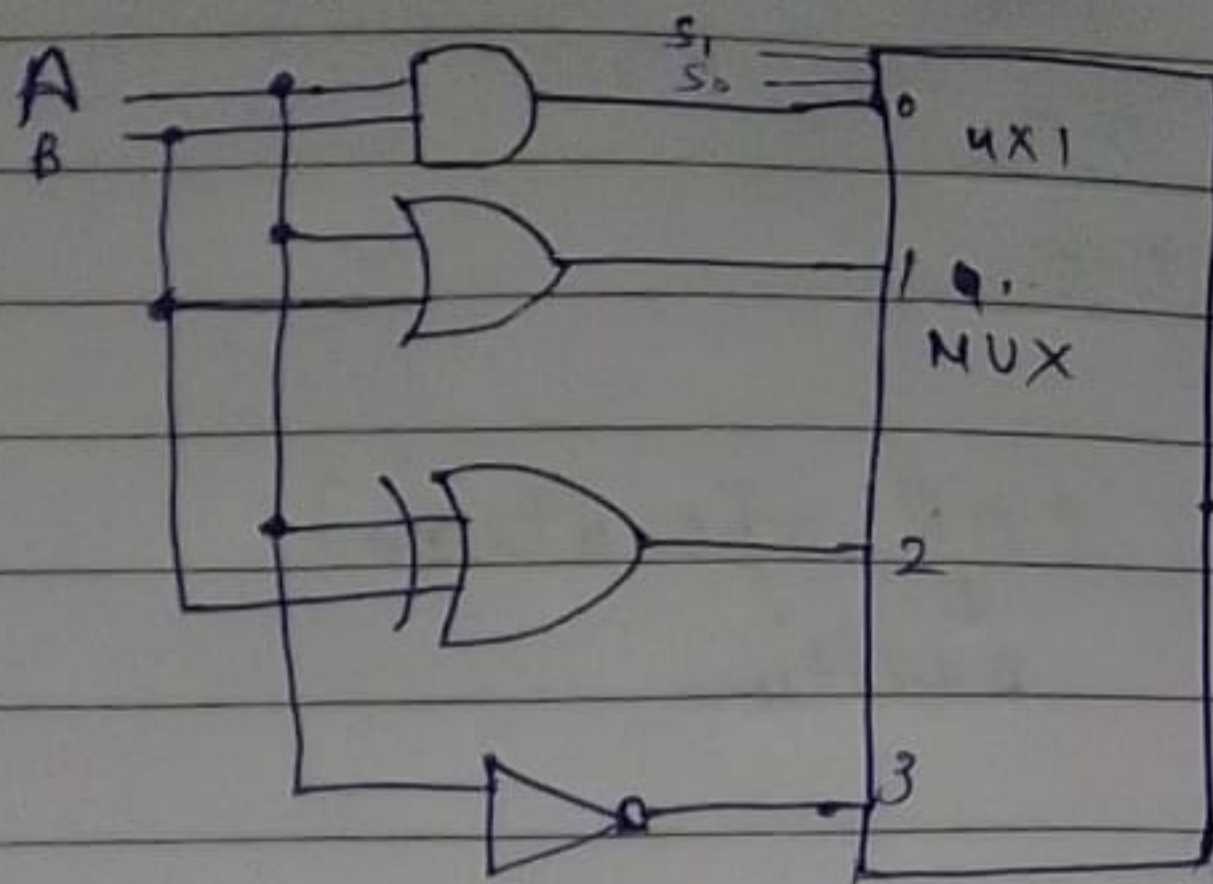
$f \leftarrow \text{All is}$

x	y	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	0	0	1

Hardware Implementation

Demultiplexer = $1 \times 4 \rightarrow$ output

MUX - Multiplexer - $4 \times 1 \rightarrow$ output



S_1	S_0	O/P	operation
0	0	$E = A \cap B$	AND
0	1	$E = A \cup B$	OR
1	0	$E = A \oplus B$	XOR
1	1	$E = \bar{A}$	Complement

Selective Set

$1010 \leftarrow A \rightarrow \text{Register}$
 $\underline{1100} \leftarrow \text{operand (operation perform karna)}$
 $\underline{1110} \text{ after}$

jithe B di value 1 $\rightarrow$ 1

jithe B di value 0 $\rightarrow$ value of A

Selective Complement

jithe B di value 0 $\rightarrow$ value of A (same)

jithe B di value 1 $\rightarrow$ Complement of value of A

1010
 $\underline{1100}$
 0110

Selective clear

jithe B di value 1 $\rightarrow$ 0

jithe B di value 0 $\rightarrow$ same value of A

1010
 1100
 $\underline{0010}$

multiply
↑# Mask Operations (AND operations)

$$\begin{array}{r} 1010 \\ 1100 \\ \hline 1000 \end{array}$$

Insert Operation (OR operation)
↓
Addition

$$\begin{array}{r} 1010 \\ 1100 \\ \hline 1110 \end{array}$$

Clear operation (XOR operation)

$$\begin{array}{r} 1010 \\ 1100 \\ \hline 0110 \end{array}$$

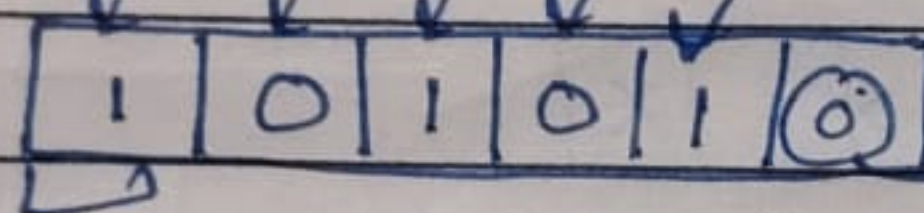
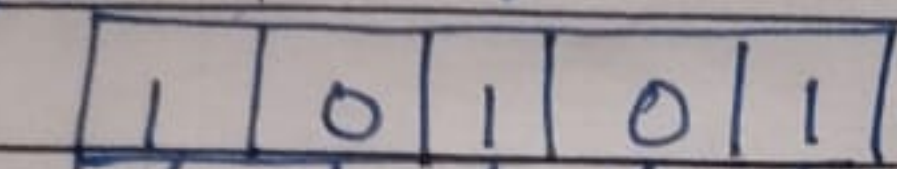
$$\begin{array}{r} 1010 \\ 1010 \\ \hline 0000 \end{array}$$

28/8/25

shifts

1. Logical Left shift

MSB LSB



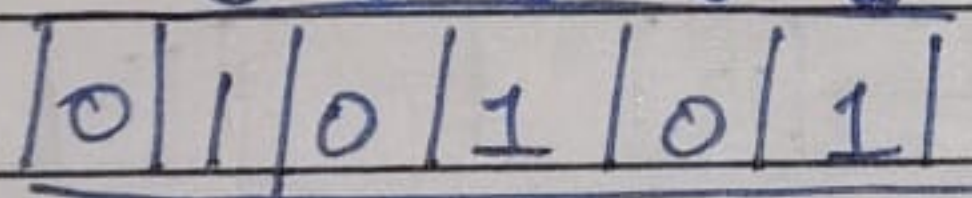
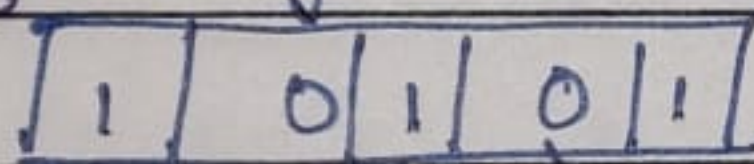
here we fill it with zero.

This bit is discarded

2. Logical Right shift

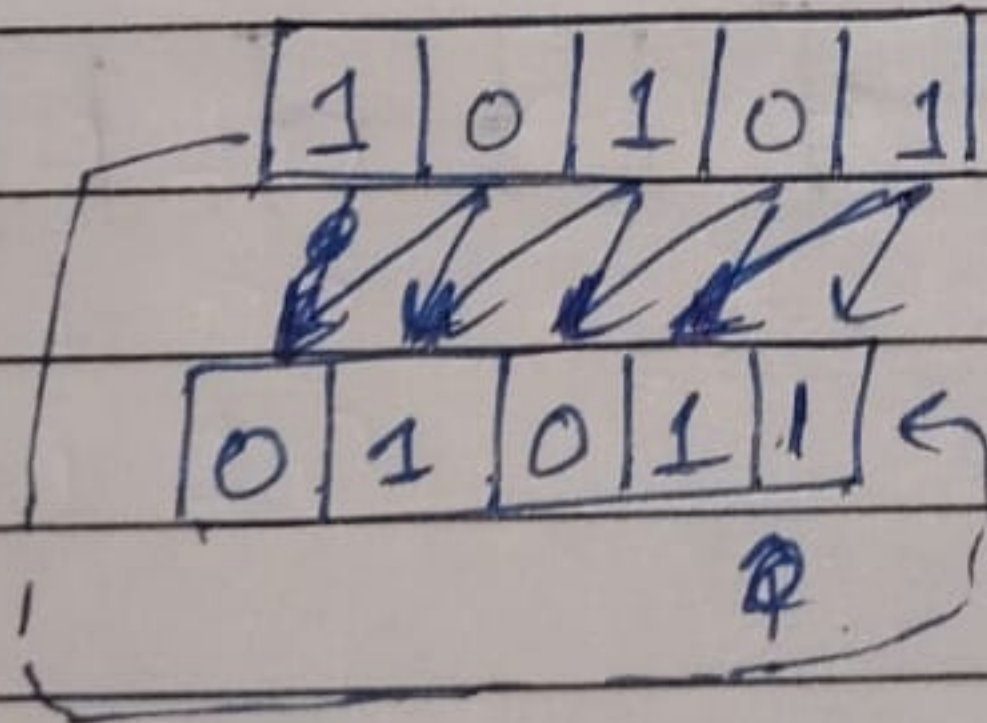
MSB

LSB

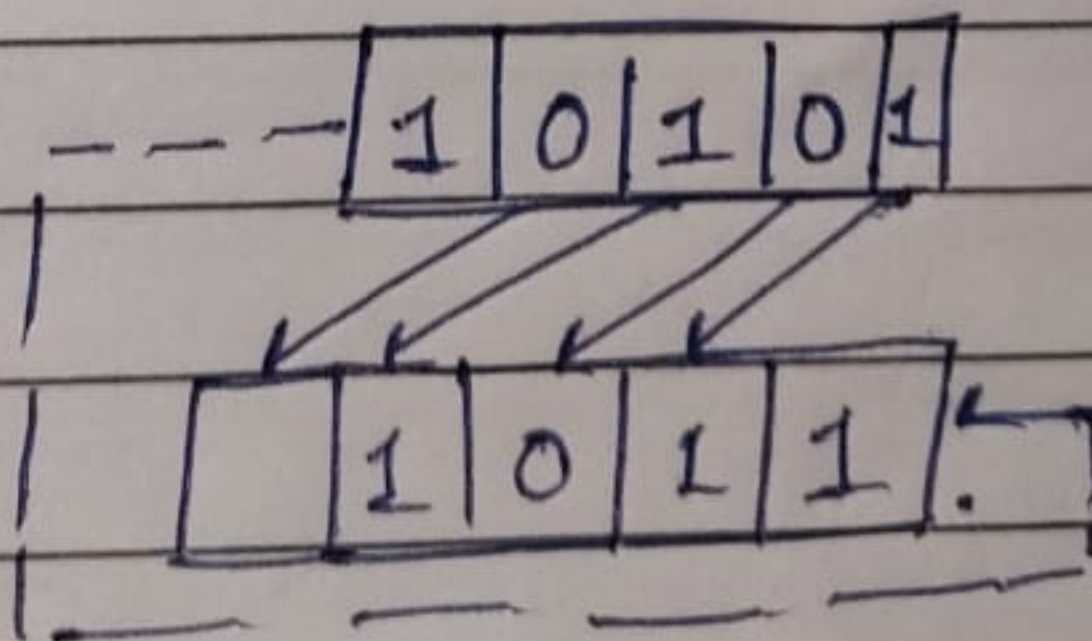


This is discarded.

3. Circular Left Shift



Replace LSB with MSB



Arithmetic Operations.

1. Addition

$$R_3 \leftarrow R_1 + R_2$$

2. Subtraction

$$R_3 \leftarrow R_1 - R_2$$

$$R_3 \quad 10 - 5$$

$$R_3 \rightarrow 5$$

3. Increment $\Rightarrow R_1 \leftarrow R_1 + 1$

4. Decrement $\Rightarrow R_1 \leftarrow R_1 - 1$

5. 1's complement

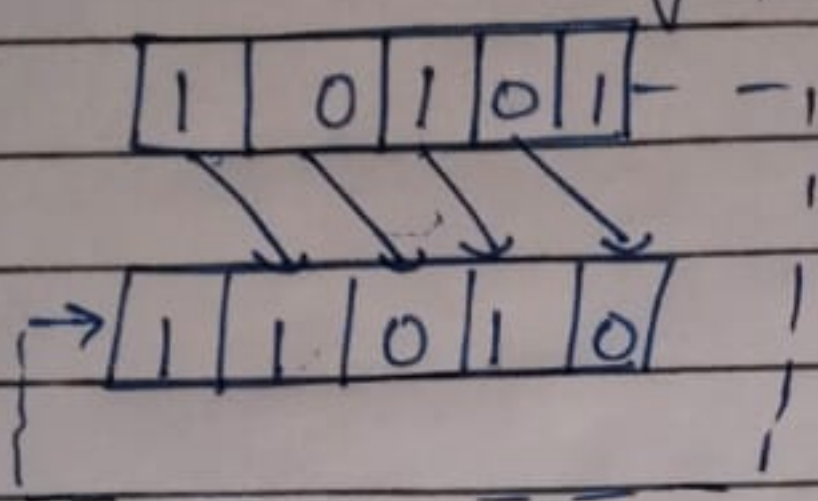
$$R_1 \leftarrow \bar{R}_1$$

6. 2's complement

$$R_2 \leftarrow \bar{R}_2 + 1$$

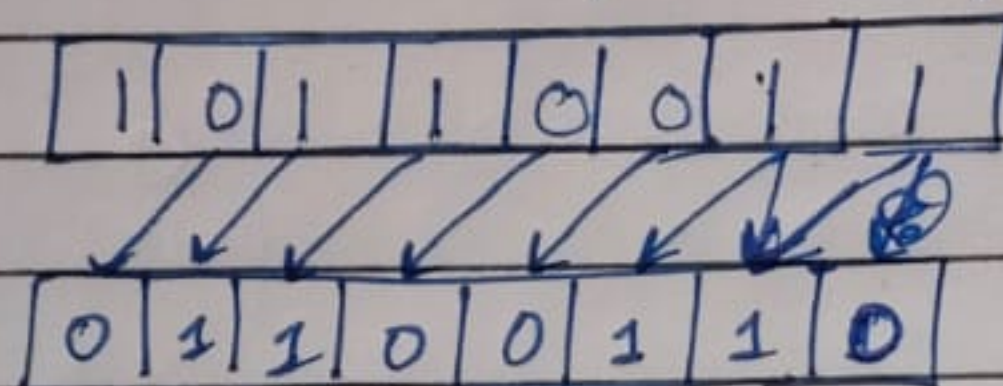
28/8/2

4. Circular right shift



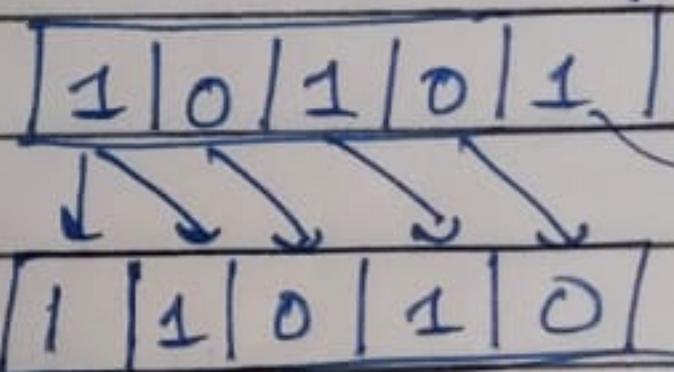
5. Arithmetic Left Shift

(use for -ve numbers)



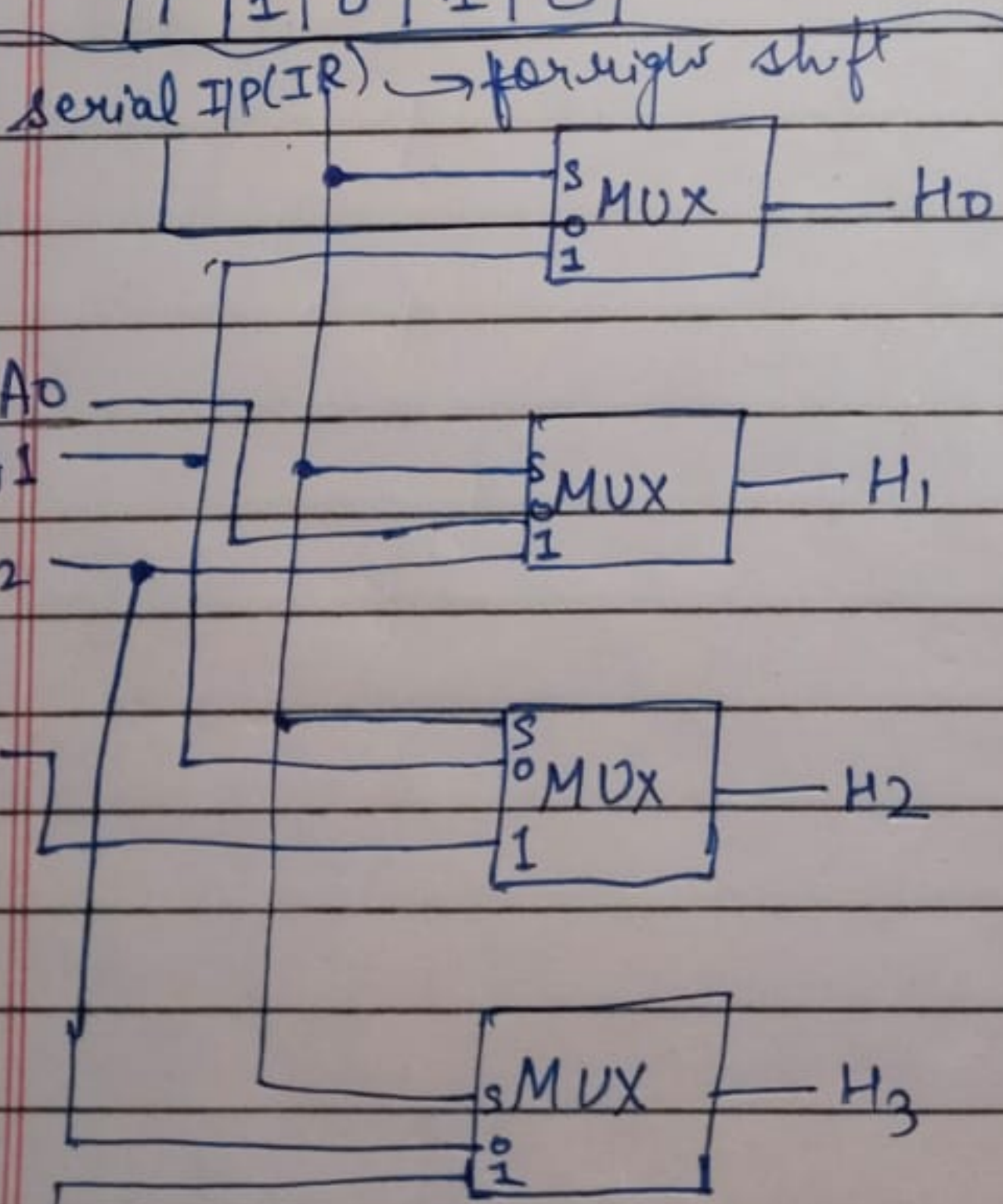
(doing left shift before)

6. Arithmetic Right Shift



→ last bit is discard.

Hardware Implementation



0 → shift Right
1 → shift Left

serial IP (I₂) → for left shift